通信协议说明

1. 接线方式

传感器使用外部 5V DC 供电，支持 UART TTL 通信方式，默认波特率 921600bps，接口连接方式定义如下。

序号	颜色	描述
1	红色	+5V
2	黑色	TTL TX
3	黄色	TTL RX
4	绿色	GND

通信关系中，传感器作为通信从机，接收并响应主机的指令。

2. 帧结构定义

```
typedef struct __attribute__((packed))
{
    uint8_t          header0;
    uint8_t          header1;
    masterCmdTyp     msgType;
    uint8_t          dataLen;
    uint8_t          data[130];  //128 + 2
}ToMasterTyp;

typedef struct __attribute__((packed))
{
    uint8_t          header0;
    uint8_t          header1;
    masterCmdTyp     msgType;
    uint8_t          dataLen;
    uint8_t          data[130]; //128 + 2
}FromMasterTyp;
```

字段	header0	Header1	msgType	dataLen	data[130]
解释	0xAA	0x55	指令类型	数据长度（仅包含有效数据部分， 帧头、消息类型和 CRC 等除外）	有效数据存储区域，动态长度，最后两个字节是 Crc0、Crc1
字节	1 字节	1 字节	1 字节	1 字节	最多 128+2 个字节

3. 指令

指令	值	描述
PC_CMD_BAUDRATE_SET	0x02	修改波特率
PC_CMD_SWVER_GET	0x03	获取固件版本
PC_CMD_SN_GET	0x04	获取传感器序列号
PC_CMD_AUTO_UPLOAD_START	0x09	使能传感器周期性发送
PC_CMD_AUTO_UPLOAD_STOP	0x0A	停止传感器周期性发送
PC_CMD_SAMPLE_RATE_SET	0x0F	修改采样率

4. 指令实例

传感器上电后，默认处于周期性自动上传检测数据的模式，后续可以通过发送停止或使能指令实现周期性自动上传检测数据的停止或恢复。

4.1 传感器周期性上传检测数据（30 个字节，单方向）

```
AA 55 01 18 69 13 2B 42 A7 34 02 43 CF 1C CB C2 EB 46 40 40 A2 79 D4 BE
C5 6D 61 41 95 17
```

“AA 55”：消息头；

“01”：指令类型；

“18”：数据区长度（0x34）；

“69 13 2B 42 A7 34 02 43 CF 1C CB C2 EB 46 40 40 A2 79 D4 BE C5 6D 61 41”：数据区，传感器的测量数据，6 个 float，小端模式；

“95 17”：CRC16，计算范围从 byte0~byte27，不包含 CRC 的两个字节，大端模式；

4.2 使能周期性自动发送指令

```
AA 55 09 00 37 BC
```

“AA 55”：消息头；

“09”：指令类型；

“00”：数据区长度为 0；

“37 BC”：CRC16，计算范围从 byte0 开始，不包含 CRC 的两个字节，大端模式；

传感器反馈

```
AA 55 09 01 00 FC 46
```

“AA 55”：消息头；

“09”：指令类型，返回主机指令；

“01”：数据区长度；

“00”：数据区，保留；

“FC 46”：CRC16，计算范围从 byte0 开始，不包含 CRC 的两个字节，大端模式；

4.3 停止周期性自动发送指令

AA 55 0A 00 37 4C

传感器反馈

AA 55 0A 01 00 0C 46

4.4 读取主板固件版本

AA 55 03 00 31 1C

“AA 55”：消息头；

“03”：指令类型；

“00”：数据区长度为 0；

“31 1C”：CRC16，计算范围从 byte0 开始，不包含 CRC 的两个字节，大端模式；

传感器反馈（版本号 0.1.0，最高 255.255.255）

AA 55 03 03 00 01 00 58 DB

“AA 55”：消息头；

“03”：指令类型，返回主机指令；

“03”：数据区长度；

“00 01 00”：数据区，(V0.1.0)；

“58 DB”：CRC16，计算范围从 byte0 开始，不包含 CRC 的两个字节，大端模式；

4.5 读取传感器序列号

AA 55 04 00 33 2C

传感器反馈（最长 32 个字节）

AA 55 04 20 46 36 2D 30 32 30 2D 30 35 30 2D 46 54 4A 31 2D 32 38 00 00 00 00 00 00 00 00 00 00 00 00 00 00 AC 6F

4.6 修改波特率为 115200bps 指令

AA 55 02 01 00 8D 84

传感器反馈（最多重复三次）

AA 55 02 02 03 00 B4 95

4.7 修改波特率为 230400bps 指令

AA 55 02 01 01 4C 44

传感器反馈（最多重复三次）

AA 55 02 02 03 00 B4 95

4.8 修改波特率为 460800bps 指令

AA 55 02 01 02 0C 45

传感器反馈（最多重复三次）

AA 55 02 02 03 00 B4 95

4.9 修改波特率为 921600bps 指令

AA 55 02 01 03 CD 85

传感器反馈（最多重复三次）

AA 55 02 02 03 00 B4 95

4.10 修改采样率为 500Hz

AA 55 0F 01 04 1D 84

传感器反馈（最多重复三次）

AA 55 0F 02 03 00 B6 39

4.11 修改采样率为 1000Hz

AA 55 0F 01 05 DC 44

传感器反馈（最多重复三次）

AA 55 0F 02 03 00 B6 39

5. CRC 校验计算公式

```
// ModBus CRC16
```

```
static const uint8_t s_CRCHi[] = {
    0x00, 0xC1, 0x81, 0x40, 0x01, 0xC0, 0x80, 0x41, 0x01, 0xC0,
    0x80, 0x41, 0x00, 0xC1, 0x81, 0x40, 0x01, 0xC0, 0x80, 0x41,
    0x00, 0xC1, 0x81, 0x40, 0x00, 0xC1, 0x81, 0x40, 0x01, 0xC0,
    0x80, 0x41, 0x01, 0xC0, 0x80, 0x41, 0x00, 0xC1, 0x81, 0x40,
    0x00, 0xC1, 0x81, 0x40, 0x01, 0xC0, 0x80, 0x41, 0x00, 0xC1,
    0x81, 0x40, 0x01, 0xC0, 0x80, 0x41, 0x01, 0xC0, 0x80, 0x41,
    0x00, 0xC1, 0x81, 0x40, 0x01, 0xC0, 0x80, 0x41, 0x00, 0xC1,
    0x81, 0x40, 0x00, 0xC1, 0x81, 0x40, 0x01, 0xC0, 0x80, 0x41,
    0x00, 0xC1, 0x81, 0x40, 0x01, 0xC0, 0x80, 0x41, 0x01, 0xC0,
    0x80, 0x41, 0x00, 0xC1, 0x81, 0x40, 0x00, 0xC1, 0x81, 0x40,
    0x01, 0xC0, 0x80, 0x41, 0x01, 0xC0, 0x80, 0x41, 0x00, 0xC1,
    0x81, 0x40, 0x01, 0xC0, 0x80, 0x41, 0x00, 0xC1, 0x81, 0x40,
    0x00, 0xC1, 0x81, 0x40, 0x01, 0xC0, 0x80, 0x41, 0x01, 0xC0,
    0x80, 0x41, 0x00, 0xC1, 0x81, 0x40, 0x00, 0xC1, 0x81, 0x40,
    0x01, 0xC0, 0x80, 0x41, 0x00, 0xC1, 0x81, 0x40, 0x01, 0xC0,
    0x80, 0x41, 0x01, 0xC0, 0x80, 0x41, 0x00, 0xC1, 0x81, 0x40,
    0x00, 0xC1, 0x81, 0x40, 0x01, 0xC0, 0x80, 0x41, 0x01, 0xC0,
    0x80, 0x41, 0x00, 0xC1, 0x81, 0x40, 0x01, 0xC0, 0x80, 0x41,
    0x00, 0xC1, 0x81, 0x40, 0x00, 0xC1, 0x81, 0x40, 0x01, 0xC0,
```

```
    0x80, 0x41, 0x00, 0xC1, 0x81, 0x40, 0x01, 0xC0, 0x80, 0x41,
    0x01, 0xC0, 0x80, 0x41, 0x00, 0xC1, 0x81, 0x40, 0x01, 0xC0,
    0x80, 0x41, 0x00, 0xC1, 0x81, 0x40, 0x00, 0xC1, 0x81, 0x40,
    0x01, 0xC0, 0x80, 0x41, 0x01, 0xC0, 0x80, 0x41, 0x00, 0xC1,
    0x81, 0x40, 0x00, 0xC1, 0x81, 0x40, 0x01, 0xC0, 0x80, 0x41,
    0x00, 0xC1, 0x81, 0x40, 0x01, 0xC0, 0x80, 0x41, 0x01, 0xC0,
    0x80, 0x41, 0x00, 0xC1, 0x81, 0x40
} ;
// CRC
static const uint8_t s_CRCLo[] = {
    0x00, 0xC0, 0xC1, 0x01, 0xC3, 0x03, 0x02, 0xC2, 0xC6, 0x06,
    0x07, 0xC7, 0x05, 0xC5, 0xC4, 0x04, 0xCC, 0x0C, 0x0D, 0xCD,
    0x0F, 0xCF, 0xCE, 0x0E, 0x0A, 0xCA, 0xCB, 0x0B, 0xC9, 0x09,
    0x08, 0xC8, 0xD8, 0x18, 0x19, 0xD9, 0x1B, 0xDB, 0xDA, 0x1A,
    0x1E, 0xDE, 0xDF, 0x1F, 0xDD, 0x1D, 0x1C, 0xDC, 0x14, 0xD4,
    0xD5, 0x15, 0xD7, 0x17, 0x16, 0xD6, 0xD2, 0x12, 0x13, 0xD3,
    0x11, 0xD1, 0xD0, 0x10, 0xF0, 0x30, 0x31, 0xF1, 0x33, 0xF3,
    0xF2, 0x32, 0x36, 0xF6, 0xF7, 0x37, 0xF5, 0x35, 0x34, 0xF4,
    0x3C, 0xFC, 0xFD, 0x3D, 0xFF, 0x3F, 0x3E, 0xFE, 0xFA, 0x3A,
    0x3B, 0xFB, 0x39, 0xF9, 0xF8, 0x38, 0x28, 0xE8, 0xE9, 0x29,
```

```
    0xEB, 0x2B, 0x2A, 0xEA, 0xEE, 0x2E, 0x2F, 0xEF, 0x2D, 0xED,
    0xEC, 0x2C, 0xE4, 0x24, 0x25, 0xE5, 0x27, 0xE7, 0xE6, 0x26,
    0x22, 0xE2, 0xE3, 0x23, 0xE1, 0x21, 0x20, 0xE0, 0xA0, 0x60,
    0x61, 0xA1, 0x63, 0xA3, 0xA2, 0x62, 0x66, 0xA6, 0xA7, 0x67,
    0xA5, 0x65, 0x64, 0xA4, 0x6C, 0xAC, 0xAD, 0x6D, 0xAF, 0x6F,
    0x6E, 0xAE, 0xAA, 0x6A, 0x6B, 0xAB, 0x69, 0xA9, 0xA8, 0x68,
    0x78, 0xB8, 0xB9, 0x79, 0xBB, 0x7B, 0x7A, 0xBA, 0xBE, 0x7E,
    0x7F, 0xBF, 0x7D, 0xBD, 0xBC, 0x7C, 0xB4, 0x74, 0x75, 0xB5,
    0x77, 0xB7, 0xB6, 0x76, 0x72, 0xB2, 0xB3, 0x73, 0xB1, 0x71,
    0x70, 0xB0, 0x50, 0x90, 0x91, 0x51, 0x93, 0x53, 0x52, 0x92,
    0x96, 0x56, 0x57, 0x97, 0x55, 0x95, 0x94, 0x54, 0x9C, 0x5C,
    0x5D, 0x9D, 0x5F, 0x9F, 0x9E, 0x5E, 0x5A, 0x9A, 0x9B, 0x5B,
    0x99, 0x59, 0x58, 0x98, 0x88, 0x48, 0x49, 0x89, 0x4B, 0x8B,
    0x8A, 0x4A, 0x4E, 0x8E, 0x8F, 0x4F, 0x8D, 0x4D, 0x4C, 0x8C,
    0x44, 0x84, 0x85, 0x45, 0x87, 0x47, 0x46, 0x86, 0x82, 0x42,
    0x43, 0x83, 0x41, 0x81, 0x80, 0x40
};

uint16_t Crc16Calc(uint8_t *data, uint16_t length)
{
```

```
    uint8_t crcHi = 0xFF;
    uint8_t crcLo = 0xFF;
    uint16_t index;

    while (length--)
    {
        index = crcHi ^ *data++;
        crcHi = crcLo ^ s_CRCHi[index];
        crcLo = s_CRCLo[index];
    }

    return (crcHi << 8) | crcLo;
}
```